

SkillGraph: Resume Extraction and RAG Pipeline

Stage 1: PDF Pre-processing

Before Gemini sees anything, the resume needs to be converted into clean text. This step is easy to skip and consistently causes problems when skipped. Resumes arrive as design-heavy PDFs with multi-column layouts, tables for skills sections, and sometimes as scanned images. Feeding a raw PDF byte stream to an LLM produces garbage or misaligned extraction.

The tool for this is `pymupdf4llm`. In benchmarks it outputs clean structured Markdown in around 0.12 seconds, preserving section headers like "Education" and "Work Experience" as `##` headings. This structure matters a lot downstream because Gemini can anchor its extraction on section labels rather than guessing where one block ends and another begins. It also handles multi-column layouts without scrambling the reading order, which most alternatives fail at.

For scanned or image-based resumes (photographed PDFs, older formats), `pymupdf4llm` has built-in OCR fallback via Tesseract. It only triggers OCR on pages where the extractable text is insufficient, so you don't pay the OCR penalty on every document.

The preprocessing step is: detect file type, run `pymupdf4llm.to_markdown()`, check if output is above a minimum character threshold, fall back to full OCR if not, then pass the cleaned Markdown to Gemini. Not complex, but without it extraction quality is unpredictable.

Stage 2: Structured Extraction with Gemini and Instructor

The extraction agent takes the cleaned Markdown and returns a validated `ResumeProfile` JSON. Two things are critical here: the prompting strategy and the output validation layer.

Prompting. The CoT approach works, but it needs to be two-stage rather than single-step. The first pass asks Gemini to reason through the resume section by section and identify skills with supporting evidence from the text. The second pass formats that reasoning into the target JSON schema. Single-step extraction (prompt + schema in one call) consistently misses implied skills and produces lower-confidence level assignments. Two-stage costs one extra Gemini call per profile but the quality difference is significant enough to justify it, especially given that the skill profile feeds every downstream agent.

The target schema per skill looks like: `name`, `category`, `level` (one of beginner/intermediate/advanced), `evidence` (a short string from the resume text that justifies the level assignment), and `years` (inferred from date ranges where present). The

`evidence` field is particularly important because it feeds the GitHub-less fallback path for skill credibility scoring.

Validation. Gemini's JSON output is inconsistent at edges. Missing fields, wrong types, occasionally wrapped in prose. The fix is `instructor`, a library that wraps any LLM client and adds Pydantic-backed auto-retry: when the output fails schema validation, it automatically re-prompts Gemini with the specific validation error. It supports Gemini natively. This should be in the architecture diagram as a structural component, not just a risk mitigation note. Without it, the extraction agent will produce silent malformed data that corrupts everything from Gap Analysis onward.

Setting `temperature=0` on the extraction call also helps a lot. Extraction is not a creative task.

Stage 3: Embedding Model

The choice here is between `google/text-embedding-004` (free, via AI Studio, 768-dim) and `BAAI/bge-large-en-v1.5` (open-source, self-hosted on the Cloud Run instance already in the stack).

For the current free-tier constraint and the fact that everything else in the stack is already Google-side, `text-embedding-004` is the right default. It performs well on long documents (JD chunks can run 500-800 tokens) and the free tier limit under AI Studio is generous enough for the dev phase and moderate production usage. If that limit becomes a problem, `bge-large-en-v1.5` is a clean swap with comparable accuracy and no API dependency.

One note on embedding the knowledge base vs. embedding queries: use the `RETRIEVAL_DOCUMENT` task type when embedding the Pinecone corpus and `RETRIEVAL_QUERY` when embedding skill lookup queries. Google's model is fine-tuned differently for these two task types and using the wrong one measurably hurts retrieval precision.

Stage 4: RAG Pipeline for Gap Analysis

This is where a lot of RAG systems underperform, and the gap between a working prototype and a reliable production system is usually here. A straightforward single-step semantic search against the skill taxonomy will work for common skills but fails at two specific problems: exact skill name disambiguation (PyTorch vs TensorFlow are semantically close but are different skills) and prerequisite ordering (retrieving "`neural networks`" when the user only knows "`linear algebra`" requires specific ordering logic that pure cosine similarity doesn't give you).

The recommended approach is a three-step retrieval pipeline.

Step 1: Metadata pre-filter. Every vector in Pinecone carries metadata including `role`, `skill_category`, `level_required`, and `corpus` (taxonomy / course / JD). Before running similarity search, filter by `role=<target_role>` and `corpus=taxonomy`. This removes cross-role noise immediately and makes the subsequent similarity search cheaper and more precise.

Step 2: Hybrid retrieval. After pre-filtering, run BM25 lexical search and dense vector search in parallel and merge results with weighted scoring. For skill name lookups, lean toward BM25 (exact match matters more): roughly `BM25=0.4`, `dense=0.6`. For JD corpus lookups where the query is a natural-language job requirement, flip it: `BM25=0.2`, `dense=0.8`. Pinecone's free tier supports one index, so implement BM25 via `rank_bm25` in Python as a pre-filter before the vector query, then merge the two result sets by reciprocal rank fusion. This is maybe 30 lines of Python and noticeably improves recall on technical skill names.

Step 3: Cross-encoder reranker. Take the top-20 merged results and run them through a lightweight cross-encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`). This model runs in around 50ms on CPU, reduces the candidate set to top-5, and cuts the rate of vaguely-relevant retrievals reaching Gemini. The reason this matters: Gemini's gap analysis quality is directly tied to the quality of the skill nodes it receives. Sending 20 loosely relevant nodes and asking Gemini to figure out gaps produces meandering output. Sending 5 high-precision nodes that are actually missing from the user's profile produces clean, actionable gaps.

The Planner Agent then operates on those top-5 gaps, runs a separate RAG pass over the course catalog with similar retrieval logic, and assembles the curriculum.

Stage 5: Knowledge Base Structure

Three corpora live in Pinecone under separate namespaces within the single free-tier index.

Role-Skill Taxonomy. Each record is a single skill node, not chunked. The metadata carries `prerequisites` as a JSON array of skill names. This lets the Planner Agent do prerequisite graph traversal in Python rather than RAG, which is faster and more reliable. Don't embed prerequisite dependency into the text chunk and try to retrieve it semantically. Keep it structured.

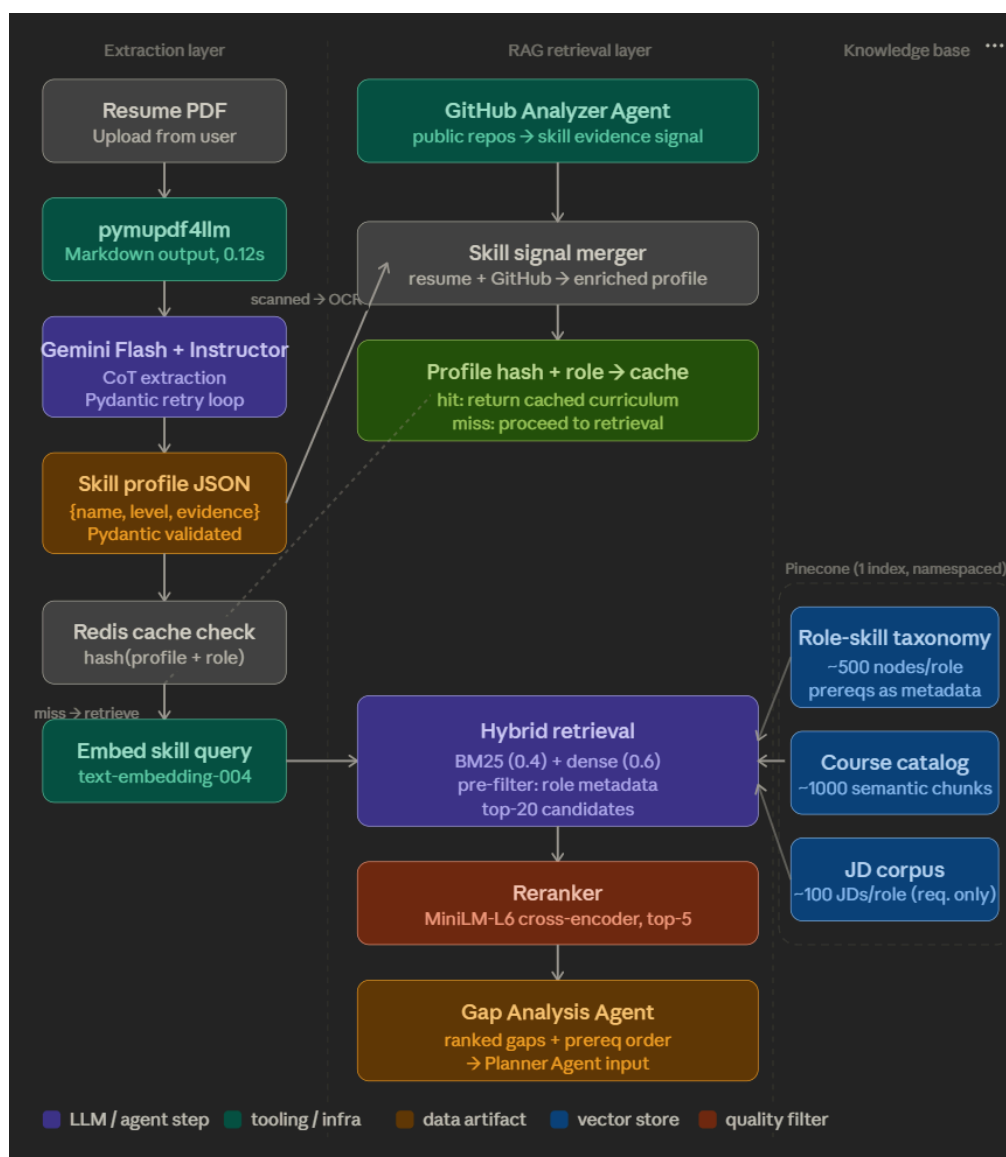
Course Catalog. Each record is one course, represented as overview + key skills covered + prerequisites. Keep it as a single chunk per course. Breaking courses into sub-chunks loses the relationship between what the course teaches and what it requires, which is exactly what the Planner Agent needs. Most course descriptions are 3-5 sentences, so this stays well within the embedding model's token limit.

JD Corpus. Strip boilerplate before embedding. Only embed the "Requirements" and "Responsibilities" sections of each JD. The company description, benefits, and "about us"

sections add semantic noise that degrades retrieval precision on skill relevance scoring. Pre-process each JD to extract these two sections before upserting to Pinecone.

Stage 6: Cache Key Strategy

The proposal has Redis caching keyed by "profile hash." One thing worth specifying: the hash should be computed over the serialized, sorted **ResumeProfile** JSON plus the target role string, not over the raw PDF bytes. Two uploads of the same resume as different files will miss the cache if you hash bytes. Hashing over structured content means the same resume re-uploaded always hits cache, and a role change correctly invalidates it.



Here Github Part is optional..although it wont be difficult to implement as it will only require to fetch one api per user and will use that one api of github to get all meta data of all public repos of user that can be used to align user's skills with user's public repos.